kt-bench: A Reproducible, Leakage-Checked Benchmark of Classic and Neural Knowledge Tracing

Jahangir · Draft, November 2026

Draft — Introduction and Discussion in progress

Abstract

Knowledge tracing predicts whether a student will answer their next question correctly from the answers they have already given. We re-implement six models behind one interface: a per-skill average, Bayesian Knowledge Tracing (BKT), Performance Factors Analysis (PFA), a Bayesian Beta-posterior counting model, Deep Knowledge Tracing (DKT) and Self-Attentive Knowledge Tracing (SAKT). We evaluate them under one protocol on ASSISTments 2009 and a 50,000-student sample of EdNet KT1. The protocol follows pyKT: student-level splits, one prediction per question, and multi-skill questions predicted "all-in-one" so no skill sees the answer to another skill of the same question. Tests enforce this, along with causality, determinism and padding. On ASSISTments 2009, averaged over five draws of the held-out students, DKT reaches 0.7575 ± 0.0045 AUC (pyKT reports 0.7541) and SAKT 0.7359 ± 0.0042 (pyKT: 0.7246). The best classic model, BKT, reaches 0.7217 ± 0.0040. DKT's lead holds for students with only 1–5 prior answers. On EdNet, tuned on the same 12-trial budget, DKT again leads (0.6776 ± 0.0002), ahead of SAKT (0.6674 ± 0.0009), PFA (0.6517 ± 0.0002) and BKT (0.6495 ± 0.0001). An untuned Beta-posterior baseline is competitive on ranking (0.7024 AUC) but poorly calibrated (ECE 0.105); a tuned prior halves the calibration error.

1. Introduction

Outline only; to be written by the author.

- The task: predict the next answer from the answer history; what it is used for (mastery estimates shown to instructors, choosing what to practise next).
- The gap: published numbers depend on protocol details (row vs. student splits, KC- vs. question-level scoring, label leakage), and small deployments need to know whether a neural model is worth it.
- What this work does: one interface, one scorer, leakage tests, five-draw uncertainty, a cold-start breakdown, cost vs. accuracy.
- Contributions (3 bullets): reproduction within ~0.003 AUC for DKT; a cold-start analysis; an open, one-command benchmark with CI-tested protocol guards.

2. Related Work

Bayesian Knowledge Tracing. Corbett and Anderson [1] model each skill as a two-state hidden Markov model (known / not known) with learn, guess and slip probabilities; the parameters are usually fit by EM. pyBKT [2] is the common open implementation.

Performance Factors Analysis. Pavlik, Cen and Koedinger [3] replace the hidden state with a logistic regression on each student's prior successes and failures per skill, which handles multi-skill items naturally.

Deep Knowledge Tracing. Piech et al. [4] feed the answer history to an LSTM and read off a probability per skill. Later work questioned how much of the original gain was real [5, 6] and showed DKT's predictions can move in the wrong direction after a correct answer [7].

Self-attention. SAKT [8] replaces the recurrence with attention from the skill being asked over embeddings of past answers; AKT [9] adds monotonic attention and Rasch-style item embeddings.

Benchmarks and protocol. pyKT [10] standardised preprocessing, student-level splits and question-level evaluation, and showed that "one-by-one" evaluation of expanded multi-skill sequences leaks labels and inflates AUC (DKT on ASSISTments 2009: 0.8262 one-by-one vs. 0.7541 question-level all-in-one).

Do we need to go deep? Gervet et al. [11] and Mandalapu et al. [12] report that well-featured logistic regression models can match deep models, especially on smaller datasets.

3. Data

ASSISTments 2009–2010 Skill Builder [13], corrected and collapsed release. The raw file matches pyKT's Table 1 exactly (4,217 students; 346,860 rows; 26,688 questions; 123 skills). Cleaning follows pyKT: drop the 63,755 rows with no skill tag (no other field is missing and every response is 0/1), then drop students with fewer than 3 answers (486 rows). This leaves 3,852 students, 282,619 question-level answers, 17,737 questions and 123 skills, or 337,424 skill-level rows. pyKT reports 337,415 skill-level rows. The 9-row difference is exactly pyKT's additional rule of dropping 200-token windows shorter than 3 tokens (5 students with 201, 202 or 602 skill-level rows). We keep and score those rows. ASSISTments 2009 has no timestamps; answers are ordered by `order_id`.

EdNet KT1 [14] holds one CSV per student with the letter chosen, not right/wrong. We join each answer to the question's correct letter and tags from the Contents release, read the CSVs directly from the zip in parallel, and write Parquet. We use a random sample of 50,000 students (seed 0); after dropping students with fewer than 3 answers and questions without tags, it has 46,951 students, 6,118,335 answers, 12,264 questions and 188 skill tags. Answers are ordered by timestamp.

4. Methods

All models implement `fit(train, valid)` and `predict(df)`. `predict` returns, for every row, P(correct) using only that student's earlier rows, and a single scorer evaluates every model.

- **Skill average.** Each skill's training-set correct rate, lightly smoothed; multi-skill questions average their skills. A floor.
- **BKT.** Per-skill two-state HMM (prior, learn, guess, slip, optional forget), fit by Baum–Welch with random restarts; prediction is forward filtering. Implemented here because pyBKT 1.4.3 does not run under numpy 2 and current scikit-learn. Sequences are padded in length-sorted chunks so that EdNet fits in memory.
- **PFA.** Logistic regression on per-skill intercepts and per-skill prior success and failure counts (raw or log1p).
- **Beta posterior.** Per student and skill, a Beta(α, β) posterior; every answer adds a recency-weighted count to α (right) or β (wrong), with weight 2^(−age / half-life), and the posterior mean is the prediction. The *default* variant uses a Beta(1, 1) prior and no tuning (on EdNet, a 23.1-day half-life; on ASSISTments, no decay). The *tuned* variant also chooses a skill-centred prior strength and the half-life.
- **DKT.** Embedding of (skill, right/wrong), one-layer LSTM, a linear map to one logit per skill; the logit of the skill asked is the prediction.
- **SAKT.** Queries are embeddings of the skill asked; keys and values are embeddings of past (skill, right/wrong) answers plus position; multi-head attention with a causal mask, residual connections, layer norm and a feed-forward block.

Multi-skill questions (leakage guard). Sequence models read one token per (question, skill). Each token carries `ctx`, the position where its question starts. DKT reads the LSTM state at `ctx`. SAKT may attend only to positions up to `ctx`, using a shifted sequence with a beginning-of-sequence token so every position has at least one key. All skills of a question are therefore predicted from the same pre-question state ("all-in-one"), and their probabilities are averaged into one question-level prediction (late fusion, mean).

5. Evaluation Protocol

- **Splits by student.** 20% of students form the test set; the other 80% are split into 5 folds. Run *r* trains on four folds, early-stops and tunes on fold *r*, and is scored on the test students.
- **Task and unit.** One-step-ahead prediction, one prediction per question; each student's first answer is not scored (as in pyKT). On ASSISTments a test set has 59,446 scored answers.
- **Metrics.** AUC (headline), accuracy at 0.5, Brier score, expected calibration error (10 equal-width bins), and AUC by number of prior answers (1–5, 6–20, 21+).
- **Uncertainty.** Repeating training seeds on one fixed test set gives tiny standard deviations (≤ 0.0006 AUC) that understate the real uncertainty. The headline ASSISTments table therefore averages over five different draws of the test students (split seeds 0–4) with each model's default settings.
- **Tuning.** Random search, scored on validation fold 0 only; test students are never loaded during tuning. Every tuned model samples 12 configurations (or its whole grid when that has exactly 12 points) from the grid in Table 0. Neural models early-stop on validation AUC (patience 10 and at most 100 epochs on ASSISTments; patience 3 and at most 30 epochs on EdNet).

Table 0. Search grids (identical on both datasets unless noted).

Model	Searched	Points	Fixed
Skill average	none	–	smoothing 1
BKT	forget on/off × max guess {0.3, 0.5} × max slip {0.1, 0.3, 0.5}	12	restarts 5 (ASSISTments); 2 restarts, ≤ 30 EM iterations (EdNet)
PFA	C ∈ {0.001, 0.01, 0.1, 1, 10, 100} × counts {raw, log1p}	12	L2 logistic regression
Beta posterior (tuned)	prior {Beta(1, 1), skill-centred} × strength {1, 2, 4, 8} × half-life {none, 3, 5, 10, 20, 50 answers} (ASSISTments) or {none, 1, 3, 10, 23.1, 60 days} (EdNet)	48	unit evidence weight
DKT	hidden {100, 200} × dropout {0.1, 0.2, 0.3} × learning rate {1e-3, 5e-4}	12	embedding 200, 1 LSTM layer, batch 64 (EdNet 128), windows of 200
SAKT	embedding {64, 128, 256} × blocks {1, 2} × dropout {0.1, 0.2, 0.3} × learning rate {1e-3, 5e-4}	36	8 heads, batch 64 (EdNet 128), windows of 200

- **Guards, run in CI.** No test student appears in train or validation; changing a student's current or future answers never changes an earlier prediction (including the multi-skill case); the same seed gives the same AUC to four decimals; padded positions never affect predictions or the loss.

6. Results

Table 1. ASSISTments 2009, five draws of the test students (default settings; mean ± std over draws).

Model	AUC	pyKT AUC (ours – pyKT)	Brier	ECE	AUC, 1–5 prior answers
Skill average	0.6224 ± 0.0042	–	0.2145	0.0113	0.6335
BKT	0.7217 ± 0.0040	–	0.1912	0.0118	0.7367
PFA	0.7087 ± 0.0039	–	0.1967	0.0215	0.7098
Beta posterior (Beta(1, 1) prior)	0.7024 ± 0.0052	–	0.2086	0.1049	0.7118
Beta posterior (skill-centred prior)	0.7103 ± 0.0045	–	0.1965	0.0483	0.7249
DKT	**0.7575 ± 0.0045**	0.7541 (+0.0034)	**0.1802**	**0.0073**	**0.7639**
SAKT	0.7359 ± 0.0042	0.7246 (+0.0113)	0.1878	0.0230	0.7520

Table 1 uses every model's config defaults on every draw; the skill-centred Beta row is a hand-set variant (prior strength 2, 10-answer half-life), not a tuned one.

Table 2. ASSISTments 2009, fixed seed-42 test set, tuned settings (mean ± std over 5 training seeds). With the same default settings, the seed-42 test set is 0.0059 AUC easier than an average of the five draws (mean over all seven models), so these numbers sit slightly above Table 1.

Model	AUC	Accuracy	ECE	Train (s, median)	Inference (µs/answer)
Skill average	0.6214 ± 0.0004	0.6592	0.0080	0.3	1.6
BKT	0.7344 ± 0.0006	0.7200	0.0124	137	10.7
PFA	0.7323 ± 0.0005	0.7161	0.0101	2.0	2.3
Beta posterior (default)	0.7108 ± 0.0000	0.6927	0.1010	< 0.1	2.1
Beta posterior (tuned)	0.7152 ± 0.0002	0.7048	0.0330	0.2	2.1
DKT	0.7638 ± 0.0006	0.7340	0.0087	45	5.5
SAKT	0.7477 ± 0.0005	0.7227	0.0180	201	6.4

Table 3. EdNet KT1, 50,000-student sample, fixed test set (9,390 test students; 1,220,901 scored answers), tuned settings, mean ± std over 5 training seeds.

Model	AUC	ECE	AUC, 1–5 prior answers	Train (s, median)
Skill average	0.5905 ± 0.0002	0.0110	0.5864	10
BKT	0.6495 ± 0.0001	0.0157	0.6104	3,639
PFA	0.6517 ± 0.0002	0.0039	0.6054	91
Beta posterior (default)	0.6332 ± 0.0000	0.0462	0.5402	< 0.1
Beta posterior (tuned)	0.6412 ± 0.0002	0.0192	0.5902	10
DKT	0.6776 ± 0.0002	0.0099	0.6435	419
SAKT	0.6674 ± 0.0009	0.0260	0.6439	672

Figures A1–A4 (appendix) show calibration, AUC by number of prior answers, and training time vs. AUC.

Observations.

1. *Reproduction.* Over five test draws, DKT is within 0.0034 AUC of pyKT and SAKT is 0.0113 above it, just outside the 0.01 target.
2. *Neural vs. classic.* DKT beats the best classic model by about 0.036 AUC on ASSISTments (0.7575 vs. 0.7217 for BKT, five draws) and by about 0.026 on EdNet (0.6776 vs. 0.6517 for PFA).

3. *Cold start*. For answers with only 1–5 prior answers, DKT still leads on ASSISTments (0.7639 vs. 0.7367 for BKT). On EdNet both neural models are far ahead of the counting models (DKT 0.6435, SAKT 0.6439, BKT 0.6104, Beta default 0.5402).
4. *Calibration*. Every model other than the two Beta variants has ECE ≤ 0.026. The default Beta(1, 1) prior is badly calibrated on ASSISTments (ECE 0.10): it starts every skill at 0.5 while the base rate is about 0.66. A skill-centred prior cuts ECE to 0.033–0.048.
5. *Cost*. The Beta posterior trains in under a second and PFA in 2 s; DKT costs about 45 s per run on ASSISTments and SAKT about 201 s on a laptop GPU, for the largest accuracy gains. BKT, fit on CPU, is the slowest model on EdNet (about an hour per run, vs. 7 minutes for DKT). Inference is a few microseconds per answer for all models.

7. Discussion

Outline only; to be written by the author.

- What the size of the DKT–BKT gap means in practice; whether 0.036 AUC changes decisions.
- Why counting models struggle at cold start on EdNet but not on ASSISTments (skill granularity, 188 tags vs. 123 skills, timestamps).
- Calibration matters more than AUC when probabilities are shown to people; the prior is the cheapest fix.
- DKT's inconsistent updates (mastery of one skill drops after answering another) seen in the demo; link to [7].
- When a simple model is the right choice: tiny data, interpretability, cost.

8. Limitations

- **Default vs. tuned settings.** Table 1 uses each model's config defaults on every draw, while Table 2 uses settings tuned on validation fold 0 of the seed-42 split. Every tuned model received 12 trials on both datasets (the grids are in Table 0). Tuning does not always carry over to the test students: on EdNet, tuned SAKT (0.6674 ± 0.0009) is 0.0013 AUC below its defaults (0.6687 ± 0.0004), while tuning lifted BKT from 0.6423 to 0.6495.
- **SAKT is 0.011 AUC above pyKT.** Possible causes: our all-in-one attention mask is also used in training; test-time windows overlap by half (each scored answer has at least 100 tokens of context) instead of being cut independently; different hyperparameters. Not yet isolated.
- **Fixed test set is easier than average.** Seed 42 is 0.0059 AUC easier than an average draw, which inflates Table 2 relative to Table 1.
- **EdNet sample.** Results use a 50,000-student random sample, not the full 784,309 students.
- **EdNet license.** EdNet is CC BY-NC 4.0: research and non-commercial use only. Models trained on it must not ship in a commercial product.
- **No real course data.** No data from a live course or deployment was evaluated; all results are on public benchmarks.
- **Timing.** Training times are medians of 5 runs on one laptop (RTX 4060 laptop GPU, 12 CPU threads). Some runs overlapped with other jobs (EdNet BKT ran on CPU alongside the GPU queue), and the laptop slept during one EdNet BKT run (586 minutes vs. 47–67 for the other four) and one DKT tuning trial, so times are indicative only; the median is used to limit their effect.
- **Scope.** No AKT or other newer attention models; no item (question-ID) embeddings in any model.

9. Future Work

- Add AKT and simpleKT, the strongest models in the pyKT benchmark.

- Isolate the SAKT gap: rerun with pyKT's non-overlapping test windows and its training mask.
- Full EdNet (784k students) on a larger GPU.
- Per-draw tuning for the five-draw table.
- Item-level features (question IDs) for PFA and the neural models.
- Evaluate on small real-world course data with a within-student temporal split and bootstrap intervals.

References

[1] A. T. Corbett and J. R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1995.

[2] Badrinath, Wang, Pardos. pyBKT: An Accessible Python Library of Bayesian Knowledge Tracing Models. *EDM*, 2021. arXiv:2105.00385.

[3] P. I. Pavlik Jr., H. Cen and K. R. Koedinger. Performance Factors Analysis – A new alternative to knowledge tracing. *AIED*, 2009.

[4] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas and J. Sohl-Dickstein. Deep knowledge tracing. *NeurIPS*, 2015.

[5] M. Khajah, R. V. Lindsey and M. C. Mozer. How deep is knowledge tracing? *EDM*, 2016.

[6] Xiong, Zhao, Van Inwegen, Beck. Going Deeper with Deep Knowledge Tracing. *EDM*, 2016, pp. 545–550.

[7] C.-K. Yeung and D.-Y. Yeung. Addressing two problems in deep knowledge tracing via prediction-consistent regularization. *L@S*, 2018.

[8] S. Pandey and G. Karypis. A self-attentive model for knowledge tracing. *EDM*, 2019.

[9] A. Ghosh, N. Heffernan and A. S. Lan. Context-aware attentive knowledge tracing. *KDD*, 2020.

[10] Z. Liu, Q. Liu, J. Chen, S. Huang, J. Tang and W. Luo. pyKT: A Python library to benchmark deep learning based knowledge tracing models. *NeurIPS Datasets and Benchmarks*, 2022. arXiv:2206.11460.

[11] Gervet, Koedinger, Schneider, Mitchell. When Is Deep Learning the Best Approach to Knowledge Tracing? *Journal of Educational Data Mining*, 12(3):31–54, 2020.

[12] Mandalapu, Gong, Chen. Do we need to go Deep? Knowledge Tracing with Big Data. arXiv:2101.08349, 2021.

[13] M. Feng, N. Heffernan and K. Koedinger. Addressing the assessment challenge with an online system that tutors as it assesses. *User Modeling and User-Adapted Interaction*, 19(3):243–266, 2009.

[14] Choi, Lee, Shin, Cho, Park, Lee, Baek, Bae, Kim, Heo. EdNet: A Large-Scale Hierarchical Dataset in Education. arXiv:1912.03072.

Appendix: Figures

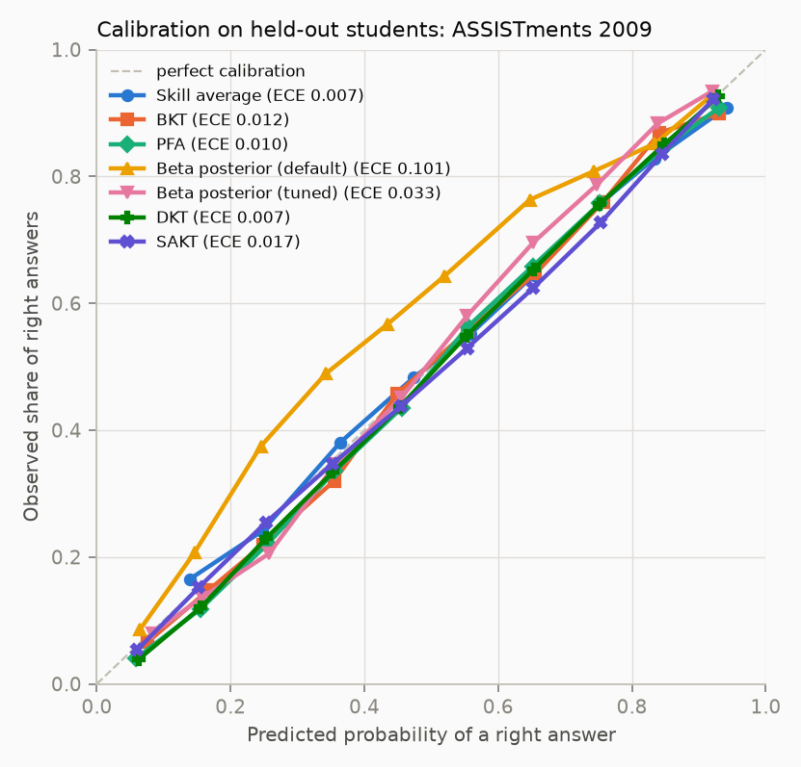


Figure A1. Calibration on held-out students, ASSISTments 2009 (seed-42 test set, all five training runs pooled).

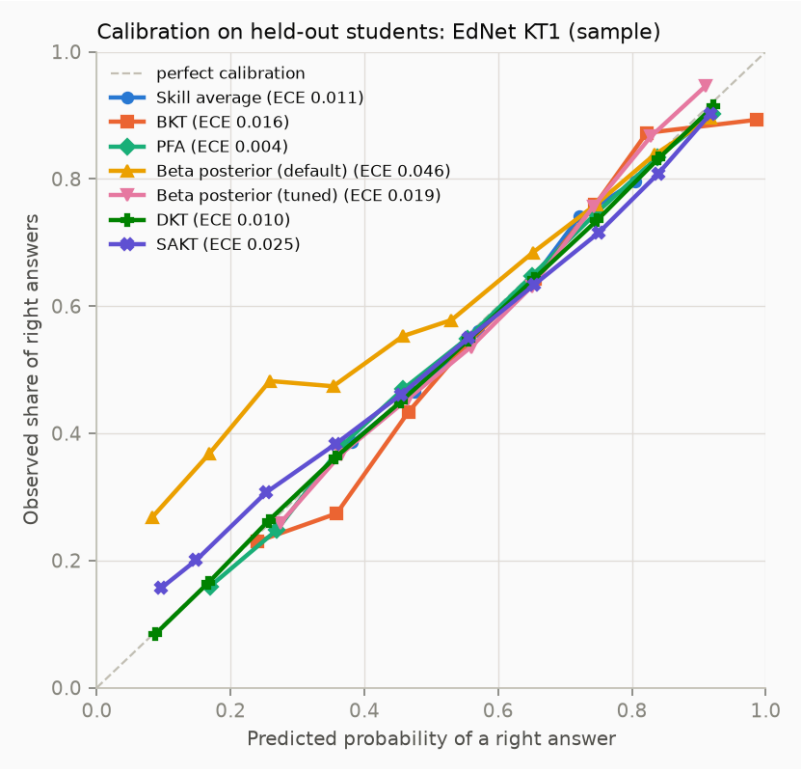


Figure A2. Calibration on held-out students, EdNet KT1 sample.

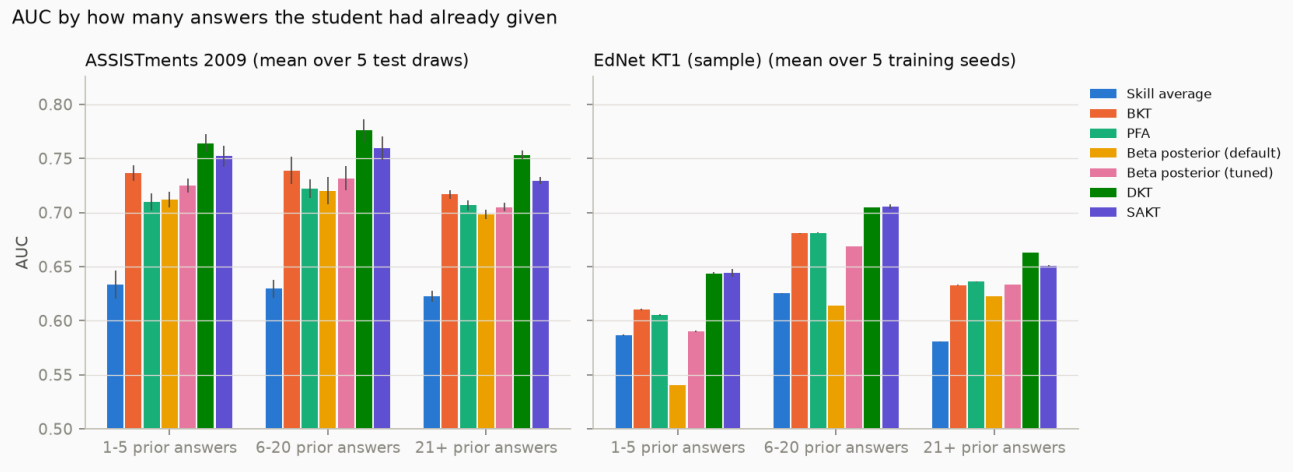


Figure A3. AUC by how many answers the student had already given.

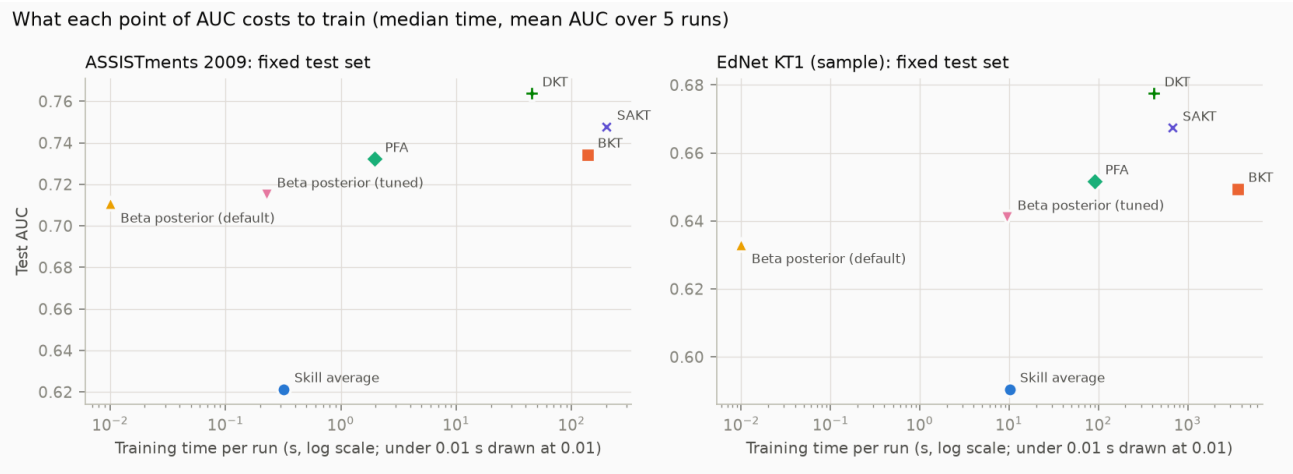


Figure A4. Training time per run (median of 5 runs, log axis) vs. mean test AUC, fixed test sets.